CS-302 Artificial Intelligence Experiment # 7

Understanding of ANN functions and learning rules

Objective/s:

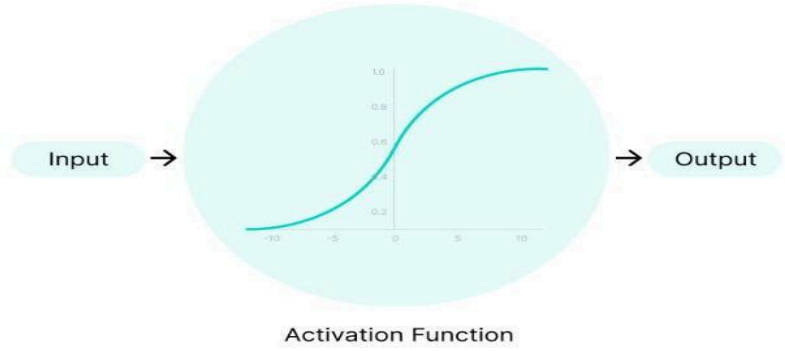
- Activation functions
- Working of ANN
- Example
- Lab Tasks

1. Activation functions in Artificial Neural Network

The activation functions play a major role in determining the output of the functions.

1.1 Why do Neural Networks need it?

Well, the purpose of an activation function is to add non-linearity to the neural network.



Activation functions introduce an additional step at each layer during the forward propagation, but its computation is worth it. Here is why—
Let's suppose we have a neural network working without the activation functions.

In that case, every neuron will only be performing a linear transformation on the inputs using the weights and biases. It's because it doesn't matter how many hidden layers we attach in the neural network; all layers will behave in the same way because the composition of two linear functions is a linear function itself.

Although the neural network becomes simpler, learning any complex task is impossible, and our model would be just a linear regression model.

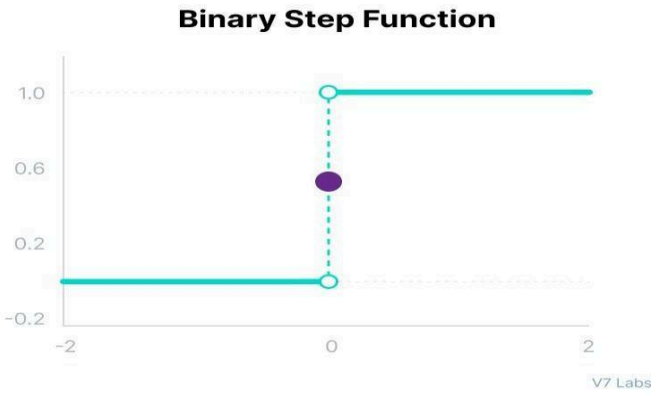
2. Types of Neural Networks Activation Functions

Now, as we've covered the essential concepts, let's go over the most popular neural networks activation

functions.

2.1 Binary Step Function

Binary step function depends on a threshold value that decides whether a neuron should be activated or not. The input fed to the activation function is compared to a certain threshold; if the input is greater than it, then the neuron is activated, else it is deactivated, meaning that its output is not passed on to the next hidden layer.

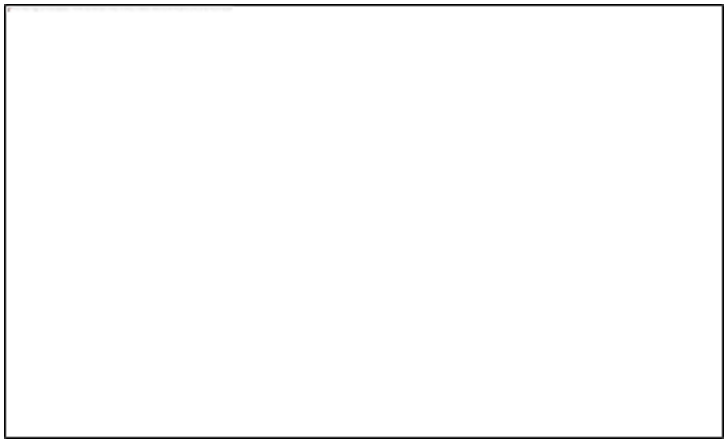


Mathematically it can be represented as:

$$\text{Binary step} \qquad f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geqslant 0 \end{cases}$$

2.2 Linear Activation Function

The linear activation function is also known as Identity Function where the activation is proportional to the input.



Mathematically it can be represented as:

$$Linear$$

$$f(x) = x$$

2.3 Non-Linear Activation Functions

The linear activation function shown above is simply a linear regression model. Because of its limited power, this does not allow the model to create complex mappings between the network's inputs and outputs.

Non-linear activation functions solve the following limitations of linear activation functions:

- They allow back propagation because now the derivative function would be related to the input, and it's possible to go back and understand which weights in the input neurons can provide a better prediction.
- They allow the stacking of multiple layers of neurons as the output would now be a non-linear combination of input passed through multiple layers. Any output can be represented as a functional computation in a neural network.

2.3.1 Sigmoid / Logistic Activation Function

This function takes any real value as input and outputs values in the range of 0 to 1. The larger the input (more positive), the closer the output value will be to 1.0, whereas the smaller the input (more negative), the closer the output will be to 0.0.

Mathematically it can be represented as:

$$Sigmoid\ /\ Logistic$$

$$f(x) = \frac{1}{1 + e^{-x}}$$

2.3.2 Tanh Function (Hyperbolic Tangent)

Tanh function is very similar to the sigmoid/logistic activation function, and even has the same S-shape with the difference in output range of -1 to 1. In Tanh, the larger the input (more positive), the closer the output value will be to 1.0, whereas the smaller the input (more negative), the closer the output will be to -1.0.

Mathematically it can be represented as:

$$Tanh$$

$$f(x) = \frac{(e^{x} - e^{-x})}{(e^{x} + e^{-x})}$$

2.3.3 ReLU Function

ReLU stands for Rectified Linear Unit. Although it gives an impression of a linear function, ReLU has a derivative function and allows for backpropagation while simultaneously making it computationally efficient. The main catch here is that the ReLU function does not activate all the neurons at the same time. The neurons will only be deactivated if the output of the linear transformation is less than 0.

Mathematically it can be represented as:

$$ReLU$$

$$f(x) = max\ (0, x)$$

if x>0:
return x
else:
return 0

2.3.4 Exponential Linear Units (ELUs) Function

Exponential Linear Unit, or ELU for short, is also a variant of ReLU that modifies the slope of the negative part of the function. ELU uses a log curve to define the negative values unlike the leaky ReLU and Parametric ReLU functions with a straight line.

Mathematically it can be represented as:

$$ELU$$

$$\begin{cases} x & for\ x \geqslant 0 \\ \alpha(e^x - 1) & for\ x < 0 \end{cases}$$

2.3.5 Softmax Function

Before exploring the ins and outs of the Softmax activation function, we should focus on its building block—the sigmoid/logistic activation function that works on calculating probability values.

Mathematically it can be represented as:

Softmax

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

3. Tasks

1. Write a user defined Python function to generate the following explained non-linear activation functions that are being used in neural networks using basic equations. Also plot them showing grid lines, title and xlabel. Use axis square.

Note: You can check your results with the results/graphs with the following image as well.


Neural Network Activation Functions
Binary Step Function
Linear
ReLU
Tanh
Sigmoid / Logistic
Leaky ReLU
max(0.1 * x,x)
Parametric ReLU
f(y)
f(y) = y
y
f(y)= ay
SELU
ELU
ELU (a=1) + Derivative
GELU
Swish